

Superior String Splitting

Document #: P2210R0
Date: 2020-08-13
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

§ 1 Introduction

Let's say you have a string like "1.2.3.4" and you want it turn it into a range of integers. You might expect to be able to write:

```
std::string s = "1.2.3.4";

auto ints =
  s | views::split('.')
    | views::transform([](auto v){
        int i = 0;
        std::from_chars(v.data(), v.size(), &i);
        return i;
    })
```

But that doesn't work. Nor does this:

```
std::string s = "1.2.3.4";

auto ints =
  s | views::split('.')
    | views::transform([](auto v){
        return std::stoi(std::string(v.begin(), v.end()));
    });
}
```

although for a different reason.

The problem ultimately is that splitting a string using C++20's `views::split` gives you a range that is *only* a forward range. Even though the source range is contiguous! And that forward range isn't a common range either. As a result, can't use `from_chars` (it needs a pointer) and can't construct a `std::string` (its range constructor takes an iterator pair, not iterator/sentinel).

The reason it's like this is that `views::split` is maximally lazy. It kind of needs to be in order to support splitting an input range. But the intent of laziness is that you can build more eager algorithms on top of them. But with this particular one, that's actually... still kind of hard. Building a less-lazy- but-still-lazy `split` on top of `views::split` can't really work. Consider:

```
std::string input = "1.2.3.4";
auto parts = input | views::split('.');

auto f = ranges::begin(parts);
auto l = ranges::end(parts);
auto n = std::next(f);
```

Let's say we actually had a hypothetical `f.base()` (`split_view`'s iterators do not provide this at the moment). That would point to the `1`, while the same hypothetical `n.base()` would point to the `2`. That's all well and good, but that means that `subrange(f.base(), n.base())` would be the range `"1."`. That's too long. We'd need to back up. But that, in of itself, only works if we have a bidirectional range - so can't even have a range of subranges for splitting a forward range. But even then, what if the pattern weren't just a single char? The iterator would need to keep track of the beginning of the previous delimiter? I'm not sure how that would work at all.

In any case, the problem here is that two points work against us:

1. `split` is overwhelmingly likely to be used on, specifically, a contiguous range of characters.
2. splitting a string is a common operation to want to do.

While I've transformed and filtered all sorts of other kinds of ranges, and have appreciated the work that went into making them as flexible as they are, I've really never wanted to split anything other than a string (or a `string_view` or other equivalent types). But for the most common use case of a fairly common operation, the `views::split` that we have ends up falling short because of what it gives us: a forward range. Pretty much every interesting string algorithm requires more than that:

- The aforementioned `std::from_chars` requires a contiguous range (really, specifically, a `char const*` and a length)
- `std::regex` and `boost::regex` both require a bidirectional range. CTRE requires a random access range.
- Many text and unicode algorithms require a bidirectional range.

It would be great if `views::split` could work in such a way that the result was maximally usable - which in this case means that splitting a contiguous range should provide contiguous sub-ranges.

§ 2 Design

I wrote a blog on this topic [[revzin.split](#)], which implements a version of `split_view` that operates on contiguous ranges (and naughtily partially specializes `std::ranges::split_view`) such that the following work:

```
auto ip = "127.0.0.1"s;
auto parts = ip | std::views::split('.');
auto as_vec = std::vector<std::string>(
    parts.begin(), parts.end());
```

as well as:

```

struct zstring_sentinel {
    bool operator==(char const* p) const {
        return *p == '\0';
    }
};

struct zstring : view_interface<zstring> {
    char const* p = nullptr;
    zstring() = default;
    zstring(char const* p) : p(p) { }
    auto begin() const { return p; }
    auto end() const { return zstring_sentinel{}; }
};

char const* words = "A quick brown fox";
for (std::string_view sv : zstring{words} | std::views::split(' ')) {
    std::cout << sv << '\n';
}

```

There are three big questions to be resolved here:

§ 2.1 What should the reference type of this be?

At a first go, given a contiguous range `V`, we could have a reference type of `span<remove_reference_t<range_reference_t<V>>>`. That is, splitting a string `const&` would yield `span<char const>`s, while splitting a vector<int> would yield `span<int>`s. This would work great.

To make the above work, we could additionally favor the `char` case. Since, again, splitting strings is the overwhelmingly common case. So we could do something like:

```

using underlying = remove_reference_t<range_reference_t<V>>>;

struct reference : span<underlying> {
    using span<underlying>::span;

    operator string_view() const
        requires same_as<range_value_t<V>, char>
    {
        return {this->data(), this->size()};
    }
};

```

Although if we actually adopt [\[P1391R4\]](#) this becomes less of an issue, since treating the reference as if it were `string_view` would just work.

§ 2.2 What category of ranges should yield this kind of value type?

As mentioned earlier, there are many useful algorithms which don't *require* contiguity, that nevertheless require something stronger than a forward range. Should splitting a random access range give you random access sub-ranges? Should splitting a bidirectional range give you bidirectional sub-ranges? The answer should be facially yes. The major selling point of Ranges is precisely this iterator category preservation to the extent that it is possible to preserve. It is unfortunate that `split` does not do so.

As mentioned earlier, I pretty much only split strings, so I care about the contiguous case much more than I care about the bidirectional case. However, if we're going to draw a line somewhere, I think the line that makes the most sense is actually between input range and forward range - let the input range be maximally laxy and have the forward case and better produce subranges (in which case the previous section can be thought of to use subranges as well rather than spans).

§ 2.3 How would `const` iteration work?

However, the big issue for moving forward with `std::ranges::split_view` is how to square some constraints:

1. `begin()` must be amortized constant time to model range
2. We can't modify things in `const` member functions in the standard library
3. The existing `split_view` is `const`-iterable.

So how do we get the first piece? What I did as part of my implementation was to not allow `const`-iteration. `split_view` just has an optional `<iterator>` member which is populated the first time you call `begin` and then is just returned thereafter. This is similar to other views that need to do arbitrary work to yield the first element (canonically, `filter_view`). See also [\[issue385\]](#). This, in a vacuum, is fairly straightforward. But, as I noted, the *existing* `split_view` is `const`-iterable - because it's lazy! It doesn't yield subranges, it yields a lazy range. So it doesn't actually need to do work in `begin()` - this is a non-issue.

This is the hardest question - since any kind of implementation that eagerly produces `span<char const>s` for splitting a string `const` necessarily has to do work to get that first span and that ends up being a clash with the existing design.

The question is - how much code currently exists that iterates over, specifically, a `const` `split_view` that is splitting a contiguous range?

Personally, I think the trade-off is hugely in favor of making this change - it makes `split` substantially more useful. But it is worth considering.

§ 3 Proposal

This paper proposes redesigning `split_view` in the following ways:

1. Splitting a range that is forward-or-better should yield subranges that are specializations of subrange (and adopting P1391 would resolve the convertibility-to-string_view issue). Splitting an input range can preserve status quo behavior.
2. `split_view` will no longer be `const`-iterable. Even though splitting an input range can preserve this functionality, I think consistency of the functionality is more important.

This could certainly break some C++20 code. But I would argue that `views::split` is so unergonomic for its most common intended use-case that the benefit of making it actually usable for that case far

outweighs the cost of potentially breaking some code.

§ 3.1 Implementation

The implementation can be found in action here [\[revzin.split.impl\]](#), but reproduced here for clarity. This implementation is strictly for contiguous ranges and produces a reference type that is a span which is conditionally convertible to `string_view`, which differs from the proposal but not in any way that's particularly interesting.

```
using namespace std::ranges;

template <contiguous_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
        std::indirectly_comparable<iterator_t<V>,
                                    iterator_t<Pattern>,
                                    equal_to>
class contig_split_view
    : public view_interface<contig_split_view<V, Pattern>>
{
public:
    contig_split_view() = default;
    contig_split_view(V base, Pattern pattern)
        : base_(base)
        , pattern_(pattern)
    { }

    template <contiguous_range R>
        requires std::constructible_from<V, views::all_t<R>>
            && std::constructible_from<Pattern, single_view<range_value_t<R>>>
    contig_split_view(R&& r, range_value_t<R> elem)
        : base_(std::views::all(std::forward<R>(r)))
        , pattern_(std::move(elem))
    { }

    struct sentinel;
    struct as_sentinel_t { };

    class iterator {
    private:
        using underlying = std::remove_reference_t<
            range_reference_t<V>>;
        friend sentinel;

        contig_split_view* parent = nullptr;
        iterator_t<V> cur = iterator_t<V>();
        iterator_t<V> next = iterator_t<V>();

    public:
        iterator() = default;
        iterator(contig_split_view* p)
            : parent(p)
            , cur(std::ranges::begin(p->base_))
            , next(lookup_next())
        { }
    };
};
```

```

iterator(as_sentinel_t, contig_split_view* p)
    : parent(p)
    , cur(std::ranges::end(p->base_))
    , next()
{ }

using iterator_category = std::forward_iterator_tag;

struct reference : std::span<underlying> {
    using std::span<underlying>::span;

    operator std::string_view() const
        requires std::same_as<range_value_t<V>, char>
    {
        return {this->data(), this->size()};
    }
};

using value_type = reference;
using difference_type = std::ptrdiff_t;

bool operator==(iterator const& rhs) const {
    return cur == rhs.cur;
}

auto lookup_next() const -> iterator_t<V> {
    return std::ranges::search(
        subrange(cur, std::ranges::end(parent->base_)),
        parent->pattern_
    ).begin();
}

auto operator++() -> iterator& {
    cur = next;
    if (cur != std::ranges::end(parent->base_)) {
        cur += distance(parent->pattern_);
        next = lookup_next();
    }
    return *this;
}

auto operator++(int) -> iterator {
    auto tmp = *this;
    ++*this;
    return tmp;
}

auto operator*() const -> reference {
    return {cur, next};
}

};

struct sentinel {
    bool operator==(iterator const& rhs) const {
        return rhs.cur == sentinel;
    }
};

```

```

    }

    sentinel_t<V> sentinel;
};

auto begin() -> iterator {
    if (not cached_begin_) {
        cached_begin_.emplace(this);
    }
    return *cached_begin_;
}
auto end() -> sentinel {
    return {std::ranges::end(base_)};
}

auto end() -> iterator requires common_range<V> {
    return {as_sentinel_t(), this};
}

private:
    V base_ = V();
    Pattern pattern_ = Pattern();
    std::optional<iterator> cached_begin_;
};

// It's okay if you're just writing a paper?
namespace std::ranges {
    template<contiguous_range V, forward_range Pattern>
    requires view<V> && view<Pattern>
        && indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, equal_to>
    class split_view<V, Pattern> : public contig_split_view<V, Pattern>
    {
        using contig_split_view<V, Pattern>::contig_split_view;
    };
}

```

§ 4 References

[issue385] Casey Carter. 2016. `const`-ness of view operations.

<https://github.com/ericniebler/range-v3/issues/385>

[P1391R4] Corentin Jabot. 2019. Range constructor for `std::string_view`.

<https://wg21.link/p1391r4>

[revzin.split] Barry Revzin. 2020. Implementing a better `views::split`.

<https://brevzin.github.io/c++/2020/07/06/split-view/>

[revzin.split.impl] Barry Revzin. 2020. Implementation of `contig_split_view`.

<https://godbolt.org/z/nyWW3F>